



دانشگاه اصفهان
دانشکده مهندسی کامپیوتر

گزارش پروژه داده کاوی

Smart Home Assistant

پدیدآورنده:

محمد امین کیانی

4003613052

رادمهر آقاخانی

4003663002

دانشجویان کارشناسی، دانشکده کامپیوتر، دانشگاه اصفهان، اصفهان،

استاد درس: جناب آقای دکتر کیانی

نیمسال دوم تحصیلی 1403-04

فهرست مطالب

3	مستندات
3	بخش اول: نصب و استفاده از کتابخانه‌ها
4	بخش دوم: اسکریپت ضبط صدا
8	بخش سوم: CustomSmartHomeAgent - هسته پردازش هوشمند
16	بخش چهارم: مدل‌ها و توابع STT/TTS
19	بخش پنجم: تعامل کامل صوتی در Colab
21	بخش ششم: Flask + Ngrok + رابط کاربری

مستندات

بخش‌های اصلی پروژه

1. نصب و ایمپورت کتابخانه‌ها
2. ضبط صوت با HTML+JS
3. Agent هوشمند برای تفسیر دستور (CustomSmartHomeAgent)
4. مدل‌های Whisper و TTS و توابع اصلی پردازش
5. تعامل صوتی کامل
6. وب‌سرور Flask با ngrok و رابط کاربری HTML

بخش اول: نصب و استفاده از کتابخانه‌ها

```
# Cell 1: Installs and Imports
!pip install --upgrade pip # Ensure pip is up-to-date
!pip install -U --force-reinstall openai-whisper TTS webrtcvad # Force
reinstall these
!pip install -U --force-reinstall ffmpeg-python # Force reinstall this
!pip install -U --force-reinstall together googletrans==4.0.0rc1 requests
# Force reinstall these
# The force-reinstall flag will ensure that dependencies like numpy and
numba are also re-evaluated and potentially reinstalled
import os
import numpy as np
from IPython.display import HTML, Audio, display
from google.colab.output import eval_js
from base64 import b64decode
import json # For the agent
import requests # For the agent
from googletrans import Translator # For the agent
from together import Together # For the agent

# Standard Whisper and TTS imports
import whisper
from TTS.api import TTS

print("All necessary libraries installed/imported.")
```

whisper-1: مدل صوتی تشخیص گفتار برای STT یا تبدیل صوت به متن

TTS-2: برای تبدیل متن به صوت

together-3: برای ارتباط با API مدل LLaMA3

Together AI بستری کامل برای چرخه‌ی زندگی کامل مدل‌های هوش مصنوعی مولد است؛ یعنی:

- استفاده از مدل‌های از پیش آموزش‌دیده (Pre-trained)
 - شخصی‌سازی و فاین‌تیون مدل‌ها
 - ساخت مدل‌های اختصاصی از پایه
 - آموزش مدل‌های بزرگ با GPUهای قدرتمند
 - اجرا و استقرار سریع مدل‌ها روی ابر یا VPC
 - با تضمین‌های امنیتی و سازگاری با استانداردهای سازمانی
- Googletrans-4: برای ترجمه خودکار (انگلیسی ← فارسی)
- ffmpeg-python-5: برای تبدیل فرمت‌های صوتی در پایتون
- eval_js-6: برای اجرای جاوااسکریپت از طرف پایتون

بخش دوم: اسکریپت ضبط صدا

```
AUDIO_HTML_RECORDER_CORRECTED = ""
<script>
// Global variables to manage recorder state
var colab_recorder_instance = null;
var colab_stream_instance = null;
var colab_audio_chunks = [];

// These will hold the resolve/reject functions for the current Promise
var colab_resolve_audio_promise = null;
var colab_reject_audio_promise = null;

// This master function is called by Python's eval_js.
function colab_js_recordAudio() {
  return new Promise((resolve, reject) => {
    colab_resolve_audio_promise = resolve;
    colab_reject_audio_promise = reject;

    colab_audio_chunks = [];
```

```

    const statusElement =
document.getElementById("colabAudioRecordingStatus");
    const stopButton =
document.getElementById("colabStopAudioRecordingButton");

    if (statusElement) statusElement.innerText = "Initializing
microphone...";
    if (stopButton) stopButton.disabled = true;

var constraints = { audio: true, video: false };
navigator.mediaDevices.getUserMedia(constraints)
    .then(function(stream) {
        colab_stream_instance = stream;
        colab_recorder_instance = new MediaRecorder(stream, { mimeType:
'audio/webm' });

        colab_recorder_instance.ondataavailable = function(e) {
            if (e.data.size > 0) { colab_audio_chunks.push(e.data); }
        };

        colab_recorder_instance.onstop = function(e) {
            var blob = new Blob(colab_audio_chunks, { 'type': 'audio/webm'
});

            colab_audio_chunks = [];
            var reader = new FileReader();
            reader.onloadend = function() {
                var base64Audio = reader.result.split(',')[1];
                if (colab_resolve_audio_promise) {
colab_resolve_audio_promise(base64Audio); }
            };
            reader.onerror = function(error) {
                if (colab_reject_audio_promise) {
colab_reject_audio_promise("FileReader error: " + error.toString()); }
            };
            reader.readAsDataURL(blob);
            if (statusElement) statusElement.innerText = "Audio captured,
processing...";
        };

        colab_recorder_instance.onerror = function(event) {
            if (colab_reject_audio_promise) {
colab_reject_audio_promise('MediaRecorder error: ' + event.error.name + "
- " + event.error.message); }
            if (statusElement) statusElement.innerText = "Recorder error:
" + event.error.name;

```

```

    };

    colab_recorder_instance.start();
    if (statusElement) statusElement.innerText = "Recording... Click
'Stop Recording' below.";
    if (stopButton) stopButton.disabled = false;
  })
  .catch(function(err) {
    if (statusElement) statusElement.innerText = "Microphone access
error: " + err.message;
    if (stopButton) stopButton.disabled = true;
    if (colab_reject_audio_promise) {
colab_reject_audio_promise("Microphone access error: " + err.message); }
  });
});
}

function colab_js_stopRecording() {
  const statusElement =
document.getElementById("colabAudioRecordingStatus");
  const stopButton =
document.getElementById("colabStopAudioRecordingButton");
  if (colab_recorder_instance && colab_recorder_instance.state ===
"recording") {
    colab_recorder_instance.stop();
  }
  if (colab_stream_instance) {
    colab_stream_instance.getTracks().forEach(track => track.stop());
  }
  if (statusElement) statusElement.innerText = "Recording stopped by
user.";
  if (stopButton) stopButton.disabled = true;
}
</script>
<div>
  <div id="colabAudioRecordingStatus" style="margin-bottom:10px;">Python
will initiate recording.</div>
  <button id="colabStopAudioRecordingButton"
onclick="colab_js_stopRecording()" disabled style="padding:8px 12px;
background-color:#f44336; color:white; border:none; border-radius:4px;
cursor:pointer;">Stop Recording</button>
</div>
"""

```

قطعه کد HTML+JS که به کاربر اجازه داده تا صدای خود را ضبط کند. از `navigator.mediaDevices.getUserMedia` برای گرفتن میکروفون استفاده می‌کند.

- با جاوااسکریپت صدا را ضبط کند، (کد JS یک رشته‌ی چندخطی هست).
- به صورت base64 رمزگذاری کند،
- به پایتون بفرستد، (در پایتون فقط داریم یک string می‌سازیم).
- در فایل ذخیره شود تا بعداً توسط Whisper و یا سایر مدل‌ها پردازش شود و تبدیل به متن کند.

```
def record_colab_audio(filename="colab_recorded_audio.webm"):
    display(HTML(AUDIO_HTML_RECORDER_CORRECTED))
    print("Please grant microphone access in your browser if prompted.")
    print("Recording will start automatically. Speak, then click 'Stop Recording' in the UI above.")
    try:
        base64_audio_data = eval_js('colab_js_recordAudio()')
        audio_bytes = b64decode(base64_audio_data)
        with open(filename, 'wb') as f: f.write(audio_bytes)
        print(f"Audio recorded and saved as '{filename}' ({len(audio_bytes)} bytes)")
        return filename
    except Exception as e:
        print(f"Error during audio recording: {e}")
        try:
            eval_js("""
                if (typeof colab_stream_instance !== 'undefined' &&
colab_stream_instance) {
                    colab_stream_instance.getTracks().forEach(track =>
track.stop());}
                colab_resolve_audio_promise = null; colab_reject_audio_promise
= null;

                const stopBtn =
document.getElementById("colabStopAudioRecordingButton"); if(stopBtn)
stopBtn.disabled = true;
                const statusElm =
document.getElementById("colabAudioRecordingStatus"); if(statusElm)
statusElm.innerText = "Ready or error.";
            """)
        except Exception as cleanup_e: print(f"Error during JS cleanup: {cleanup_e}")
        return None

print("Colab audio recording function defined.")
```

تابع `record_colab_audio(filename)` :

- HTML قبلی را نمایش می‌دهد.
- کاربر صحبت می‌کند و بعد روی **Stop Recording** کلیک می‌کند.
- فایل ضبط‌شده را با فرمت `webm` ذخیره می‌کند و برمی‌گرداند.

بخش سوم: CustomSmartHomeAgent - هسته پردازش هوشمند

```
• import json
• from together import Together
• import requests
• from googletrans import Translator
•
• class CustomSmartHomeAgent:
•     def __init__(self,
llm_api_key="6ba1bc9d36c18b34a8e1a820d132b0b3614192ca291165e56799eff
76f03304b", primary_llm_language='en'):
•         """
•         Initializes the Smart Home Agent.
•         llm_api_key: Your API key for the chosen LLM service
(TogetherAI).
•         primary_llm_language: The language your core LLM prompts are
optimized for (e.g., 'en').
•         """
•         self.llm_api_key = llm_api_key
•         self.primary_llm_language = primary_llm_language
•         self.llm_client = Together(api_key=self.llm_api_key)
•         print(f"CustomSmartHomeAgent initialized. LLM will primarily
process in: {self.primary_llm_language}")
•
•         self.valid_devices = {
•             "lamp": ["kitchen", "bathroom", "room 1", "room 2"],
•             "ac": ["room 1", "kitchen"],
•             "tv": ["living room"]
•         }
•
•     def _is_valid_device_location(self, device, location):
•         if not device or not location:
•             return False
•         return location.lower() in [loc.lower() for loc in
self.valid_devices.get(device, [])]
```



```

•     def _detect_language(self, text):
•         if any('\u0600' <= char <= '\u06FF' for char in text):
•             return 'fa'
•         return 'en'
•
•
•     def _translate_text(self, text, source_lang, target_lang):
•         if source_lang == target_lang:
•             return text
•
•         try:
•             translator = Translator()
•             result = translator.translate(text, src=source_lang,
dest=target_lang)
•             return result.text
•         except Exception as e:
•             print(f"Translation error: {e}")
•             return f"TranslationError_{text}_to_{target_lang}"
•
•     def _get_llm_analysis(self, command_text_for_llm):
•         prompt = f"""
•         You are an AI assistant for a smart home. Analyze the user's
command.
•
•         Respond ONLY with a JSON object containing:
•         - "intent": (e.g., "control_device", "get_weather",
"get_news", "get_time", "unknown")
•         - "device": (e.g., "lamp", "ac", "tv", "none")
•         - "action": (e.g., "turn_on", "turn_off", "set_temperature",
"change_channel", "fetch", "none")
•         - "parameters": A dictionary of parameters (e.g.,
{"location": "kitchen", "temperature": 22, "topic": "technology"
•         Limit your JSON explanation to 10 words or less in each
value.}})
•
•         User command: "{command_text_for_llm}"
•         JSON Analysis:
•         """
•         print(f"\n--- Sending to LLM (in
{self.primary_llm_language}) ---")
•         # print(prompt)
•         print("--- End of Prompt ---")
•         try:
•             response = self.llm_client.chat.completions.create(
•                 model="meta-llama/Llama-3-70b-chat-hf",
•                 messages=[{"role": "user", "content": prompt}],
•                 temperature=0.5,

```

```

•         response_format={"type": "json_object"}
•     )
•     llm_output_str = response.choices[0].message.content
•     print(f"LLM Raw Output: {llm_output_str}")
•     return json.loads(llm_output_str)
•
•     except Exception as e:
•         print(f"Error calling LLM or processing response: {e}")
•         return {"intent": "error", "message": str(e)}
•
•
•     def execute_command(self, natural_language_command):
•         original_lang =
self._detect_language(natural_language_command)
•         print(f"\nOriginal command: '{natural_language_command}'
(Detected lang: {original_lang})")
•         if original_lang != self.primary_llm_language:
•             command_for_llm =
self._translate_text(natural_language_command, original_lang,
self.primary_llm_language)
•         else:
•             command_for_llm = natural_language_command
•         if not command_for_llm:
•             print("Error: Command became empty after translation
step.")
•             return self._generate_response("translation_error",
original_lang, {})
•         analysis = self._get_llm_analysis(command_for_llm)
•         print(f"LLM Analysis: {json.dumps(analysis, indent=2)}")
•         intent = analysis.get("intent")
•         device = analysis.get("device")
•         action = analysis.get("action")
•         parameters = analysis.get("parameters", {})
•         execution_result_message = "Action executed (or would be)."
•         if intent == "control_device":
•             if not self._is_valid_device_location(device,
parameters.get("location", "")):
•                 final_response =
self._generate_response("unknown_device", original_lang,
{"device_name": f"{device} in {parameters.get('location', '')}"})
•                 # print(f"Agent Final Response ({original_lang}):
{final_response}")
•                 return final_response
•                 if device == "lamp":
•                     execution_result_message =
self._control_lamp(action, parameters)
•                 elif device == "ac":

```

```

•         execution_result_message = self._control_ac(action,
parameters)
•         elif device == "tv":
•             execution_result_message = self._control_tv(action,
parameters)
•         else:
•             intent = "unknown_device"
•             parameters["device_name"] = device
•         elif intent == "get_weather":
•             execution_result_message = self._get_weather(parameters)
•         elif intent == "get_news":
•             execution_result_message = self._get_news(parameters)
•         elif intent == "get_time":
•             execution_result_message = self._get_time(parameters)
•         final_response = self._generate_response(intent,
original_lang, parameters, execution_result_message, device, action)
•         # print(f"Agent Final Response ({original_lang}):
{final_response}")
•         return final_response
•
•
•     def _generate_response(self, intent, lang, params,
execution_message="N/A", device_name=None, action_name=None):
•         response_text = ""
•         print(intent)
•         location = params.get("location", "the specified location")
•         topic = params.get("topic")
•         if lang == 'en':
•             if intent == "control_device":
•                 response_text = f"Okay, {action_name} the
{device_name} in {location}. Status: {execution_message}"
•             elif intent == "get_weather":
•                 response_text = f"Fetching weather for {location}.
Result: {execution_message}"
•             elif intent == "get_news":
•                 response_text = f"Getting news about {topic if topic
else 'general topics'}. Result: {execution_message}"
•             elif intent == "get_time":
•                 response_text = f"The current time is:
{execution_message}"
•             elif intent == "unknown_device":
•                 response_text = f"Sorry, I don't know how to control
a device named '{params.get('device_name', 'unknown device')}.'"
•             elif intent == "translation_error":
•                 response_text = "I had trouble understanding the
command after attempting translation."

```

```

•         elif intent == "error":
•             response_text = f"An internal error occurred:
{params.get('message', 'Please try again.')}}"
•         else:
•             response_text = "I'm sorry, I didn't understand that
command. Can you please rephrase?"
•         elif lang == 'fa':
•             if intent == "control_device":
•                 action_fa = {"turn_on": "روشن کردن", "turn_off":
"خاموش کردن", "set_temperature": "تنظیم دمای"}.get(action_name,
action_name)
•                 device_fa = {"lamp": "چراغ", "ac": "کولر", "tv":
"تلویزیون"}.get(device_name, device_name)
•                 location_fa = {"kitchen": "آشپزخانه", "Room 1":
"اتاق ۱"}.get(location, location)
•                 response_text = f"{action_fa} {device_fa}
در {location_fa} وضعیت: {self._translate_text(execution_message,
'en', 'fa')}}"
•                 elif intent == "get_weather":
•                     response_text = f"در حال دریافت اطلاعات آب و هوا برای
{location}. نتیجه: {self._translate_text(execution_message, 'en',
'fa')}}"
•                 elif intent == "get_news":
•                     response_text = f"در حال دریافت اخبار در مورد {topic
if topic else 'موضوعات عمومی'}. نتیجه:
{self._translate_text(execution_message, 'en', 'fa')}}"
•                 elif intent == "get_time":
•                     response_text = f"ساعت فعلی:
{self._translate_text(execution_message, 'en', 'fa')}}"
•                 elif intent == "unknown_device":
•                     response_text = f"متاسفم، من نمی دانم چگونه دستگاهی
را کنترل کنم '{params.get('device_name', 'دستگاه ناشناس')}' به نام
•                 elif intent == "translation_error":
•                     response_text = "پس از تلاش برای ترجمه، در درک دستور
مشکل داشتم."
•                 elif intent == "error":
•                     response_text = f"یک خطای داخلی رخ داد:
{params.get('message', 'لطفا دوباره تلاش کنید.')}}"
•                 else:
•                     response_text = "متاسفم، متوجه دستور شما نشدم. ممکن
است دوباره بیان کنید؟"
•
•             return response_text if response_text else "Response
generation error."
•

```

```

•     def _control_lamp(self, action, params):
•         location = params.get("location", "default_location")
•         return f"Lamp at {location} action {action} processed."
•
•     def _control_ac(self, action, params):
•         location = params.get("location", "default_location")
•         temperature = params.get("temperature")
•         return f"AC at {location} action {action} (temp:
{temperature}) processed."
•
•     def _control_tv(self, action, params):
•         location = params.get("location", "living_room")
•         channel = params.get("channel_name")
•         return f"TV at {location} action {action} (channel:
{channel}) processed."
•
•     # def _get_weather(self, params):
•     #     location = params.get("location", "current_city")
•     #     return f"Weather for {location} is sunny, 25°C (mock)."
•
•     def _get_news(self, params):
•         topic = params.get("topic", "general")
•         api_key = "9a332f8925de495f81a0ca08b526283b" # <-- Replace
with your NewsAPI.org key
•         url = "https://newsapi.org/v2/everything"
•         try:
•             response = requests.get(url, params={
•                 "q": topic,
•                 "language": "en",
•                 "sortBy": "publishedAt",
•                 "pageSize": 1,
•                 "apiKey": api_key
•             })
•             data = response.json()
•             if data.get("status") == "ok" and data.get("articles"):
•                 article = data["articles"][0]
•                 title = article.get("title", "No title")
•                 description = article.get("description", "")
•                 return f"Top news for {topic}: {title}.
{description}"
•             else:
•                 return f"No news found for topic '{topic}'."
•         except Exception as e:
•             return f"Error fetching news: {e}"
•

```

```

• def _get_time(self, params):
•     from datetime import datetime
•     return datetime.now().strftime("%Y-%m-%d %H:%M:%S")
•
• def _get_weather(self, params):
•     location = params.get("location", "Tehran")
•     # Use Open-Meteo geocoding to get latitude and longitude
•     try:
•         geo_resp = requests.get(
•             "https://geocoding-api.open-meteo.com/v1/search",
•             params={"name": location, "count": 1, "language":
"en", "format": "json"}
•         )
•         geo_data = geo_resp.json()
•         if "results" not in geo_data or not geo_data["results"]:
•             return f"Could not find location '{location}'."
•         lat = geo_data["results"][0]["latitude"]
•         lon = geo_data["results"][0]["longitude"]
•         # Get current weather
•         weather_resp = requests.get(
•             "https://api.open-meteo.com/v1/forecast",
•             params={
•                 "latitude": lat,
•                 "longitude": lon,
•                 "current_weather": True
•             }
•         )
•         weather_data = weather_resp.json()
•         if "current_weather" in weather_data:
•             temp =
weather_data["current_weather"]["temperature"]
•             wind = weather_data["current_weather"]["windspeed"]
•             weather_str = f"Weather for {location}: {temp}°C,
wind {wind} km/h."
•             return weather_str
•         else:
•             return f"Could not get weather for '{location}'."
•     except Exception as e:
•         return f"Error fetching weather: {e}"

```

این بخش وظیفه‌ی تحلیل دستورات کاربر با کمک LLM ، اجرای فرمان‌ها، و تولید پاسخ را بر عهده دارد:

1. تشخیص زبان (فارسی یا انگلیسی)

```
def _detect_language(text): ...
```

2. ترجمه متن (اگر فارسی بود ← انگلیسی)

```
def _translate_text(text, source_lang, target_lang): ...
```

3. فرستادن دستور به LLaMA برای تحلیل:

```
def _get_llm_analysis(command_text_for_llm): ...
```

این تابع prompt را به LLaMA می‌فرستد تا یک JSON تولید کند شامل:

```
{
  "intent": "control_device",
  "device": "lamp",
  "action": "turn_on",
  "parameters": {"location": "kitchen"}
}
```

4. اجرای دستور:

```
def execute_command(natural_language_command): ...
```

وظیفه:

- تشخیص زبان
- ترجمه اگر نیاز بود
- تحلیل با LLM
- اجرای دستور با توابع `_control_lamp`, `_control_ac`, `_get_weather` و...:

5. ساخت پاسخ متنی (و ترجمه آن به فارسی در صورت نیاز)

```
def _generate_response(...)
```

```
LLM_API_KEY =
"6ba1bc9d36c18b34a8e1a820d132b0b3614192ca291165e56799eff76f03304b" # This
looks like a Mistral API key
agent = CustomSmartHomeAgent(llm_api_key=LLM_API_KEY)

print("Agent initialized.")
```

حال agent یک شیء فعال است که می‌توان با آن :

```
agent.execute_command("Turn on the kitchen lamp")
```

- زبان جمله رو تشخیص می‌دهد،
- اگر لازم باشد ترجمه می‌کند،
- با LLM مشورت می‌کند،
- تصمیم می‌گیرد که چه کاری انجام بدهد،
- و جمله پاسخ تولید می‌کند.

بخش چهارم: مدل‌ها و توابع STT/TTS

```
# Cell 4: Voice Assistant Core Logic and Setup for Colab

# --- Wake Word ---
WAKE_WORD = "hey assistant" # Assuming English

# --- Whisper Model ---
print("Loading Whisper model...")
# Using "base.en" for English and faster performance in Colab
# You can change to "small.en", "medium.en" for more accuracy but slower
speed
try:
    import torch
    WHISPER_DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
    whisper_model = whisper.load_model("base.en", device=WHISPER_DEVICE)
    print(f"Whisper model 'base.en' loaded on {WHISPER_DEVICE}.")
except Exception as e:
    print(f"Error loading Whisper model: {e}. Trying on CPU.")
    try:
        WHISPER_DEVICE = "cpu"
        whisper_model = whisper.load_model("base.en",
device=WHISPER_DEVICE)
        print("Whisper model 'base.en' loaded on CPU.")
    except Exception as e_cpu:
        print(f"Could not load Whisper model on CPU either: {e_cpu}")
        whisper_model = None
```



```

# --- TTS Model ---
print("Loading TTS model...")
try:
    # Using your specified model. If this fails, common alternatives are:
    # "tts_models/en/ljspeech/vits" or "tts_models/en/vctk/vits" (multi-
    speaker)
    tts_model = TTS(model_name="tts_models/en/ljspeech/tacotron2-DDC",
progress_bar=True, gpu=False)
    print("TTS model 'tts_models/en/ljspeech/fastspeech2' loaded.")
except Exception as e:
    print(f"Error loading TTS model 'tts_models/en/ljspeech/fastspeech2':
{e}")
    print("Attempting to load a common alternative:
'tts_models/en/ljspeech/vits'")
    try:
        tts_model = TTS(model_name="tts_models/en/ljspeech/vits",
progress_bar=True, gpu=False)
        print("TTS model 'tts_models/en/ljspeech/vits' loaded.")
    except Exception as e2:
        print(f"Could not load alternative TTS model: {e2}")
        tts_model = None

# --- Adapted STT function for Colab ---
def transcribe_colab_audio(audio_file_path, model):
    if not model:
        print("Whisper model not loaded. Cannot transcribe.")
        return ""
    if not audio_file_path or not os.path.exists(audio_file_path):
        print(f"Audio file not found: {audio_file_path}")
        return ""
    try:
        # Whisper's transcribe function can take a file path directly
        # Setting language to English based on wake word and TTS model
        result = model.transcribe(audio_file_path, language='en',
fp16=False if WHISPER_DEVICE == "cpu" else True)
        return result['text']
    except Exception as e:
        print(f"Error during transcription: {e}")
        return ""

# --- Adapted TTS function for Colab ---
def speak_colab(text_to_speak, tts_instance,
output_filename="colab_assistant_output.wav"):
    if not tts_instance:
        print("TTS model not loaded. Cannot speak.")

```

```

        return None
    if not text_to_speak:
        print("No text to speak.")
        return None
    try:
        print(f"Synthesizing speech for: '{text_to_speak}'")
        tts_instance.tts_to_file(text=text_to_speak,
file_path=output_filename)
        print(f"Speech saved to {output_filename}")
        return output_filename
    except Exception as e:
        print(f"Error during TTS synthesis: {e}")
        return None

print("\nCore assistant functions defined and models loaded (if
successful).")

```

تعریف واژه بیدارگر (Wake Word) :

WAKE_WORD = "hey assistant"

این واژه ابتدای فرمان های صوتی کاربر قرار می گیرد.

: STT

`whisper_model = whisper.load_model("base.en")`

مدل Whisper از OpenAI برای تبدیل صوت به متن استفاده می شود.

: TTS

`tts_model = TTS(model_name="tts_models/en/ljspeech/tacotron2-DDC")`

مدل Tacotron2 یا جایگزین آن برای تولید صدای طبیعی از متن استفاده می شود.

تابع `transcribe_colab_audio(...)`

صوت ضبط شده را به متن تبدیل می کند.

تابع `speak_colab(...)`

متن پاسخ را به صوت تبدیل کرده و فایل wav ذخیره می کند.

بخش پنجم: تعامل کامل صوتی در Colab

```
# Cell 5: Main Interaction Cell for Colab

if not whisper_model or not tts_model or not agent:
    print("One or more critical components (Whisper, TTS, Agent) are not
    initialized. Please check previous cells for errors.")
else:
    print("--- Voice Assistant Interaction Ready ---")
    print(f"Say '{WAKE_WORD}' followed by your command when recording
    starts.")

    # 1. Record audio from Colab interface
    recorded_audio_file = record_colab_audio()

    if recorded_audio_file:
        # 2. Transcribe the audio
        print("\nTranscribing audio...")
        transcribed_text = transcribe_colab_audio(recorded_audio_file,
        whisper_model)
        print(f"Heard: '{transcribed_text}'")

        if transcribed_text:
            # 3. Check for wake word
            if WAKE_WORD in transcribed_text.lower():
                print(f"\nWake word '{WAKE_WORD}' detected!")

                # Extract command (simple extraction, assumes wake word is
                at the start)
                command_start_index =
                transcribed_text.lower().find(WAKE_WORD) + len(WAKE_WORD)
                command_text =
                transcribed_text[command_start_index:].strip()

                if command_text:
                    print(f"Command: '{command_text}'")

                    # 4. Agent processes command
                    print("\nSending command to agent...")
                    agent_response = agent.execute_command(command_text)
                    print(f"Agent response: '{agent_response}'")

                    # 5. TTS speaks the response
```

```

        if agent_response:
            speech_output_file = speak_colab(agent_response,
tts_model)

            if speech_output_file and
os.path.exists(speech_output_file):
                print("\nPlaying agent's response:")
                display(Audio(speech_output_file,
autoplay=True))
            else:
                print("Could not generate or find speech
output file.")
        else:
            print("Agent did not provide a response to
speak.")
    else:
        print("No command found after wake word.")
else:
    print(f"\nWake word '{WAKE_WORD}' not detected in the
recording.")
else:
    print("\nTranscription was empty or failed.")

    # Clean up the recorded audio file
    if os.path.exists(recorded_audio_file):
        # os.remove(recorded_audio_file) # Uncomment to automatically
delete after processing
        pass

else:
    print("\nAudio recording failed or was cancelled. No action
taken.")

print("\nRun this cell again to start a new interaction.")

```

- ضبط صدا از کاربر
- تبدیل صدا به متن با Whisper
- بررسی وجود wake word مثل "hey assistant"
- حذف wake word و استخراج دستور
- ارسال پاسخ متنی به agent
- تبدیل پاسخ به صوت با TTS و پخش آن

به دلیل محدودیت‌های Google Colab (دسترسی به میکروفون فقط در یک لحظه ممکنه)، ما نمی‌توانیم به صورت دائمی گوش کنیم». اما می‌توانیم:

1. به ضبط کوتاه اولیه بگیریم (مثلاً 2 ثانیه)،
2. چک کنیم داخل آن Wake Word هست یا نه.
3. اگر بود، به ضبط جدید واقعی بگیریم (مثلاً 10 ثانیه).
4. بفرستیم برای Agent و پاسخ بگیریم.

بخش ششم: Flask + Ngrok + رابط کاربری

```
from flask import Flask, request, jsonify, send_from_directory,
render_template_string
from flask_cors import CORS
from pyngrok import conf, ngrok
import os
import threading
from base64 import b64decode
import uuid # For unique filenames

# --- Configuration ---
# IMPORTANT: Set your ngrok authtoken if you have one for longer
sessions/more features
conf.get_default().auth_token =
"2xh4hoPpO4OFmb9T2jQXvj9Xvio_33xggZ5o2d6Rs3aTkZZqJ" # Replace with your
token if you have one

# Create a directory to store audio responses if it doesn't exist
AUDIO_OUTPUT_DIR = 'static/audio_outputs'
if not os.path.exists(AUDIO_OUTPUT_DIR):
    os.makedirs(AUDIO_OUTPUT_DIR)

app = Flask(__name__)
CORS(app) # Enable CORS for all routes

# --- HTML Template for the UI ---
# For simplicity in Colab, we embed the HTML as a string.
# In a larger project, you'd save this as index.html in a 'templates'
folder.
INDEX_HTML = """
<!DOCTYPE html>
<html lang="en">
<head>
```

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Voice Assistant Home</title>
<style>
  body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
    margin: 0;
    background-color: #f4f7f6; /* Light grayish-blue background */
    color: #333;
  }
  .container {
    background-color: #fff;
    padding: 30px 40px;
    border-radius: 12px;
    box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
    width: 100%;
    max-width: 600px;
    text-align: center;
  }
  h1 {
    color: #007bff; /* Blue accent */
    margin-bottom: 20px;
  }
  .search-bar-container {
    display: flex;
    align-items: center;
    background-color: #fff;
    border: 1px solid #dfele5;
    border-radius: 24px;
    padding: 5px 15px;
    margin-bottom: 20px;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  }
  .search-bar-container:hover, .search-bar-container:focus-within {
    box-shadow: 0 4px 8px rgba(0,0,0,0.15);
  }
  #commandInput {
    flex-grow: 1;
    border: none;
    outline: none;
    padding: 10px;
  }

```

```

        font-size: 16px;
    }
    #recordButton, #sendButton {
        background: none;
        border: none;
        cursor: pointer;
        padding: 8px;
        margin-left: 8px;
        display: flex;
        align-items: center;
        justify-content: center;
    }
    #recordButton svg, #sendButton svg {
        width: 24px;
        height: 24px;
        fill: #5f6368;
    }
    #recordButton:hover svg, #sendButton:hover svg {
        fill: #007bff;
    }
    .status, .response-area {
        margin-top: 15px;
        padding: 12px;
        border-radius: 8px;
        font-size: 0.95em;
    }
    .status {
        background-color: #e9ecef;
        color: #495057;
        min-height: 20px;
    }
    .response-area {
        background-color: #e6f7ff; /* Light blue for response */
        color: #004085; /* Darker blue text */
        min-height: 40px;
        text-align: left;
    }
    .home-screen-elements {
        margin-top: 20px;
        padding: 10px;
        border-top: 1px solid #eee;
        font-size: 0.9em;
        color: #666;
    }
    .recording {

```

```

        animation: pulse 1.5s infinite;
    }
    @keyframes pulse {
        0% { opacity: 1; }
        50% { opacity: 0.5; }
        100% { opacity: 1; }
    }
</style>
</head>
<body>
    <div class="container">
        <h1>My Smart Assistant</h1>
        <div class="search-bar-container">
            <input type="text" id="commandInput" placeholder="Ask or tell
me something...">
            <button id="recordButton" title="Record Voice">
                <svg viewBox="0 0 24 24"><path d="M12 14c1.66 0 3-1.34 3-
3V5c0-1.66-1.34-3-3-3S9 3.34 9 5v6c0 1.66 1.34 3 3 3zm5.3-3c0 3-2.54 5.1-
5.3 5.1S6.7 14 6.7 11H5c0 3.41 2.72 6.23 6 6.72V21h2v-3.28c3.28-.49 6-3.31
6-6.72h-1.7z"></path></svg>
            </button>
            <button id="sendButton" title="Send Text">
                <svg viewBox="0 0 24 24"><path d="M2.01 21L23 12 2.01 3 2
10 15 2-15 2z"></path></svg>
            </button>
        </div>
        <div class="status" id="statusArea">Ready.</div>
        <div class="response-area" id="agentResponseArea">Agent's response
will appear here.</div>
        <audio id="responseAudio" style="width: 100%; margin-top:
15px;"></audio>
        <div class="home-screen-elements">
            <p>Try commands like: "Hey assistant, turn on the kitchen
lamp" or "What's the weather in Berlin?"</p>
        </div>
    </div>

    <script>
        const commandInput = document.getElementById('commandInput');
        const recordButton = document.getElementById('recordButton');
        const sendButton = document.getElementById('sendButton');
        const statusArea = document.getElementById('statusArea');
        const agentResponseArea =
document.getElementById('agentResponseArea');
        const responseAudio = document.getElementById('responseAudio');

```



```

    let mediaRecorder;
    let audioChunks = [];
    let isRecording = false;

    // Use Browser's SpeechRecognition for STT (Alternative to sending
    audio to Python for Whisper)
    // This is simpler for a separate webpage if you don't strictly
    need Whisper for the web UI.
    // If you want to use Whisper, you'll need to send the audio blob
    to the backend.
    // For this example, I'll implement sending audio to backend to
    use your Whisper.

    recordButton.onclick = async () => {
      if (!isRecording) {
        try {
          const stream = await
navigator.mediaDevices.getUserMedia({ audio: true });
          mediaRecorder = new MediaRecorder(stream);
          mediaRecorder.ondataavailable = event => {
            audioChunks.push(event.data);
          };
          mediaRecorder.onstop = sendAudioToBackend;
          audioChunks = [];
          mediaRecorder.start();
          isRecording = true;
          statusArea.textContent = 'Recording... Click mic again
to stop.';

          recordButton.classList.add('recording');
          recordButton.title = "Stop Recording";
        } catch (err) {
          console.error("Error accessing microphone:", err);
          statusArea.textContent = 'Error: Could not access
microphone. ' + err.message;
        }
      } else {
        mediaRecorder.stop();
        isRecording = false;
        statusArea.textContent = 'Recording stopped.
Processing...';
        recordButton.classList.remove('recording');
        recordButton.title = "Record Voice";
      }
    };

```

```

    async function sendAudioToBackend() {
        statusArea.textContent = 'Processing audio...';
        const audioBlob = new Blob(audioChunks, { type: 'audio/webm'
    }); // or 'audio/wav' etc.
        audioChunks = []; // Clear for next recording

        const formData = new FormData();
        formData.append('audio_data', audioBlob,
'recorded_audio.webm');

        try {
            const response = await fetch('/process_audio_command', {
                method: 'POST',
                body: formData
            });
            const result = await response.json();
            handleBackendResponse(result);
        } catch (error) {
            console.error('Error sending audio:', error);
            statusArea.textContent = 'Error sending audio to
backend.';

            agentResponseArea.textContent = 'Error: ' + error.message;
        }
    }

    sendButton.onclick = async () => {
        const command = commandInput.value.trim();
        if (!command) {
            statusArea.textContent = 'Please type a command or
record.';

            return;
        }
        statusArea.textContent = 'Sending command: ' + command;
        agentResponseArea.textContent = 'Processing...';
        responseAudio.src = ''; // Clear previous audio

        try {
            const response = await fetch('/process_text_command', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ command: command })
            });
            const result = await response.json();
            handleBackendResponse(result);

```

```

        } catch (error) {
            console.error('Error sending text command:', error);
            statusArea.textContent = 'Error sending command to
backend.';
            agentResponseArea.textContent = 'Error: ' + error.message;
        }
    };

    commandInput.addEventListener('keypress', function (e) {
        if (e.key === 'Enter') {
            sendButton.onclick();
        }
    });

    function handleBackendResponse(result) {
        if (result.error) {
            statusArea.textContent = 'Error from agent: ' +
result.error;
            agentResponseArea.textContent = result.text_response ||
result.error;
        } else {
            statusArea.textContent = 'Agent responded.';
            agentResponseArea.textContent = result.text_response ||
'Agent processed the command.';
            if (result.transcribed_text) { // If audio was processed
                commandInput.value = result.transcribed_text;
            }
        }

        if (result.audio_url) {
            responseAudio.src = result.audio_url;
            responseAudio.play().catch(e => console.warn("Audio play
failed:", e)); // Autoplay might be blocked
        }
    }
</script>
</body>
</html>
"""

@app.route('/')
def index():
    return render_template_string(INDEX_HTML)

@app.route('/process_text_command', methods=['POST'])

```

```

def handle_text_command():
    data = request.get_json()
    command = data.get('command')

    agent_response_text = "No command received."
    audio_file_url = None
    error_msg = None

    if command:
        try:
            # Assuming agent.execute_command returns the text response
            agent_response_text = agent.execute_command(command)
            if agent_response_text:
                # Generate a unique filename for the audio output
                unique_filename = f"{str(uuid.uuid4())}.wav"
                output_path = os.path.join(AUDIO_OUTPUT_DIR,
unique_filename)

                # speak_colab should save the file to output_path
                # You might need to adjust speak_colab to accept a full
path or manage paths here
                # For this example, assuming speak_colab can take
output_path

                # Let's redefine a simpler speak for flask:
                if tts_model:
                    tts_model.tts_to_file(text=agent_response_text,
file_path=output_path)
                    if os.path.exists(output_path):
                        audio_file_url = f"/audio/{unique_filename}"
                    else:
                        agent_response_text += " (TTS audio file not
created)"
                else:
                    agent_response_text += " (TTS model not available)"
            else:
                agent_response_text = "Agent processed command but gave no
response."
        except Exception as e:
            error_msg = str(e)
            agent_response_text = f"Error processing command: {e}"

    return jsonify({
        'text_response': agent_response_text,
        'audio_url': audio_file_url,
        'error': error_msg
    })

```

```

    })

@app.route('/process_audio_command', methods=['POST'])
def handle_audio_command():
    agent_response_text = "No audio received or error in processing."
    transcribed_text_for_ui = ""
    audio_file_url = None
    error_msg = None

    if 'audio_data' not in request.files:
        return jsonify({'error': 'No audio file part in the request.'}),
400

    file = request.files['audio_data']
    if file.filename == '':
        return jsonify({'error': 'No selected audio file.'}), 400

    if file:
        temp_audio_path = os.path.join(AUDIO_OUTPUT_DIR,
f"temp_{str(uuid.uuid4())}.webm")
        file.save(temp_audio_path)

        try:
            transcribed_text = transcribe_colab_audio(temp_audio_path,
whisper_model)
            transcribed_text_for_ui = transcribed_text

            command_to_agent = transcribed_text # Or process for wake word
            # Simplified: directly use transcribed text as command
            # You can add your WAKE_WORD logic here if needed:
            # if WAKE_WORD.lower() in transcribed_text.lower():
            #     command_start_index =
transcribed_text.lower().find(WAKE_WORD.lower()) + len(WAKE_WORD)
            #     command_to_agent =
transcribed_text[command_start_index:].strip()
            # else: ... handle no wake word ...

            if command_to_agent:
                agent_response_text =
agent.execute_command(command_to_agent)
                if agent_response_text:
                    unique_filename = f"{str(uuid.uuid4())}.wav"
                    output_path = os.path.join(AUDIO_OUTPUT_DIR,
unique_filename)
                    if tts_model:

```

```

        tts_model.tts_to_file(text=agent_response_text,
file_path=output_path)
        if os.path.exists(output_path):
            audio_file_url = f"/audio/{unique_filename}"
        else:
            agent_response_text += " (TTS audio file not
created)"
        else:
            agent_response_text += " (TTS model not
available)"
        else:
            agent_response_text = "Agent processed command but
gave no response."
        else:
            agent_response_text = "Transcription resulted in empty
command."
    except Exception as e:
        error_msg = str(e)
        agent_response_text = f"Error processing audio: {e}"
    finally:
        if os.path.exists(temp_audio_path):
            os.remove(temp_audio_path)

    return jsonify({
        'text_response': agent_response_text,
        'transcribed_text': transcribed_text_for_ui,
        'audio_url': audio_file_url,
        'error': error_msg
    })

@app.route('/audio/<filename>')
def serve_audio(filename):
    return send_from_directory(AUDIO_OUTPUT_DIR, filename)

# --- Start Flask app and ngrok tunnel ---
# Kill any existing ngrok tunnels/Flask apps if you rerun this cell
try:
    if 'public_url' in globals() and ngrok.get_tunnels():
        print(f"Closing existing ngrok tunnel: {public_url}")
        ngrok.disconnect(public_url)
        ngrok.kill()
    # You might need a way to stop the Flask app thread if it's already
running

```

```

    # This is a bit tricky in Colab; often restarting the runtime is
    easiest for a clean slate.
except Exception as e:
    print(f"Error managing existing ngrok: {e}")

# Start ngrok tunnel
try:
    # Make sure pyngrok is configured with your auth token if you have one
    # conf.get_default().auth_token = "YOUR_NGROK_AUTHTOKEN" # Run this if
not done globally
    public_url = ngrok.connect(5000) # Flask default port is 5000
    print(f" * ngrok tunnel \"{public_url}\" ->
\"http://127.0.0.1:5000\")
    print(f" * Access your UI at: {public_url}")

    # Start Flask app in a separate thread
    # Use a daemon thread so it automatically closes when the Colab
cell/runtime stops
    threading.Thread(target=app.run, kwargs={'host':'0.0.0.0',
'port':5000, 'debug':False, 'use_reloader':False}, daemon=True).start()

except Exception as e:
    print(f"Error starting ngrok or Flask: {e}")
    print("Please ensure you have ngrok installed and configured if
necessary (e.g., with an authtoken for stable URLs).")
    print("You might need to provide your ngrok authtoken via
pyngrok.conf.get_default().auth_token = 'YOUR_TOKEN'")

```

Flask : برای اجرای سرور و API ها

pyngrok : برای باز کردن تونل ngrok و نمایش وبسایت Flask در اینترنت

Uuid : تولید فایل‌های صوتی با نام یکتا

CORS : اجازه می‌دهد که JS سمت کلاینت به API های Flask وصل شود

`conf.get_default().auth_token"..." =`

اگر auth token نداشته باشیم، فقط تونل ۲ ساعته می‌تواند بسازد. با توکن رایگان، تا ۸ ساعت باز می‌ماند.

بخش Flask :

- ساخت وبسرور با Flask برای دریافت فرمان کاربر (متنی یا صوتی)
- مسیر `/process_text_command` : دریافت دستور متنی از کاربر و پاسخ متنی + صوتی می‌دهد
- مسیر `/process_audio_command` : دریافت فایل صوتی، تبدیل به متن، اجرا، پاسخ‌دهی

رابط HTML :

صفحه‌ای با فرم ساده شامل:

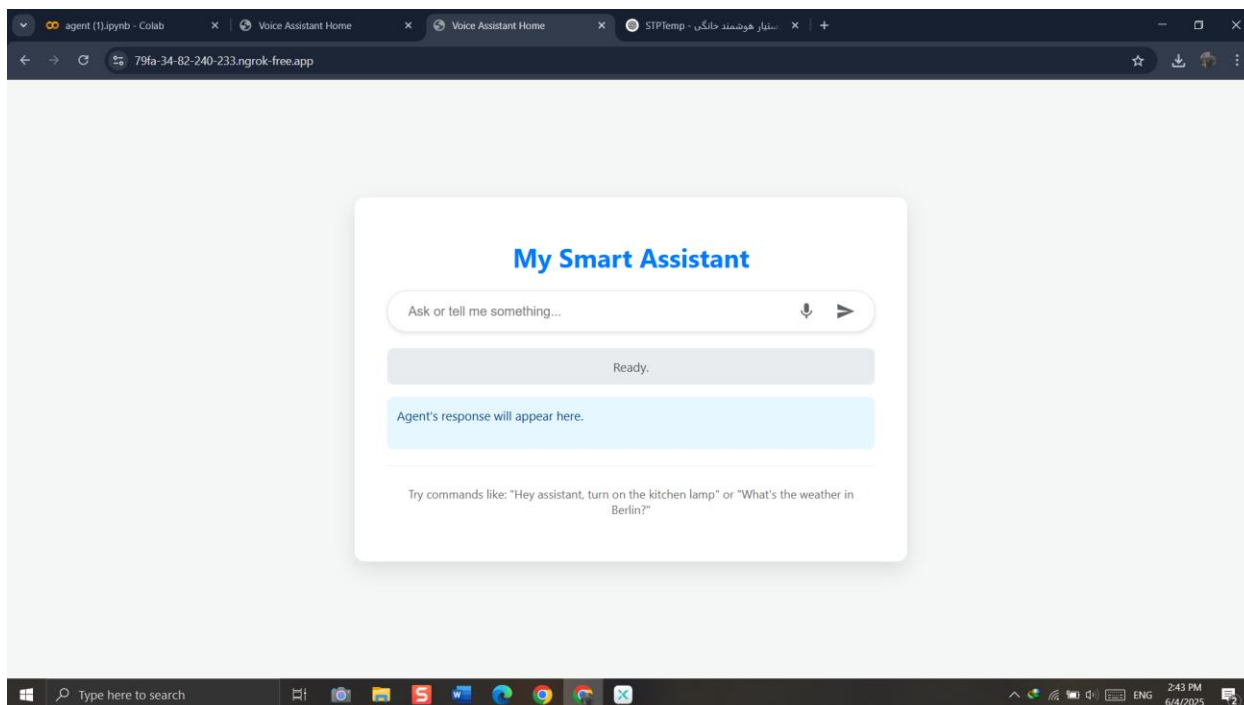
- کادر تایپ دستور
- دکمه ضبط صدا
- نمایش پاسخ دستیار
- پخش فایل صوتی خروجی

اتصال با ngrok :

یک ابزار محبوب برای تونل‌سازی شبکه است که به ما اجازه می‌دهد:

- برنامه‌های **لوکال (محلی)** خود را به‌طور موقت در اینترنت در دسترس قرار دهیم.
- از طریق **HTTPS** یا **HTTP** به برنامه‌هایی که روی `localhost` اجرا می‌شوند دسترسی پیدا کنیم. پس آدرس عمومی ایجاد تا کاربر از طریق اینترنت به این رابط دسترسی داشته باشد.
- تست API ها، Webhook ها مثل Stripe یا GitHub ، و اپلیکیشن‌های وب را ساده‌تر کنیم.

مثال: اگر برنامه‌ای روی <http://localhost:3000> داریم، ngrok آن را روی یک URL مانند <https://abcd1234.ngrok.io> قابل دسترس می‌کند.



عملکرد:

1. فایل صوتی (webm) از مرورگر دریافت می‌شود.
2. با Whisper به متن تبدیل می‌شود.
3. متن به Agent داده می‌شود.
4. خروجی Agent تبدیل به صدا می‌شود.
5. خروجی شامل:

○ پاسخ متنی

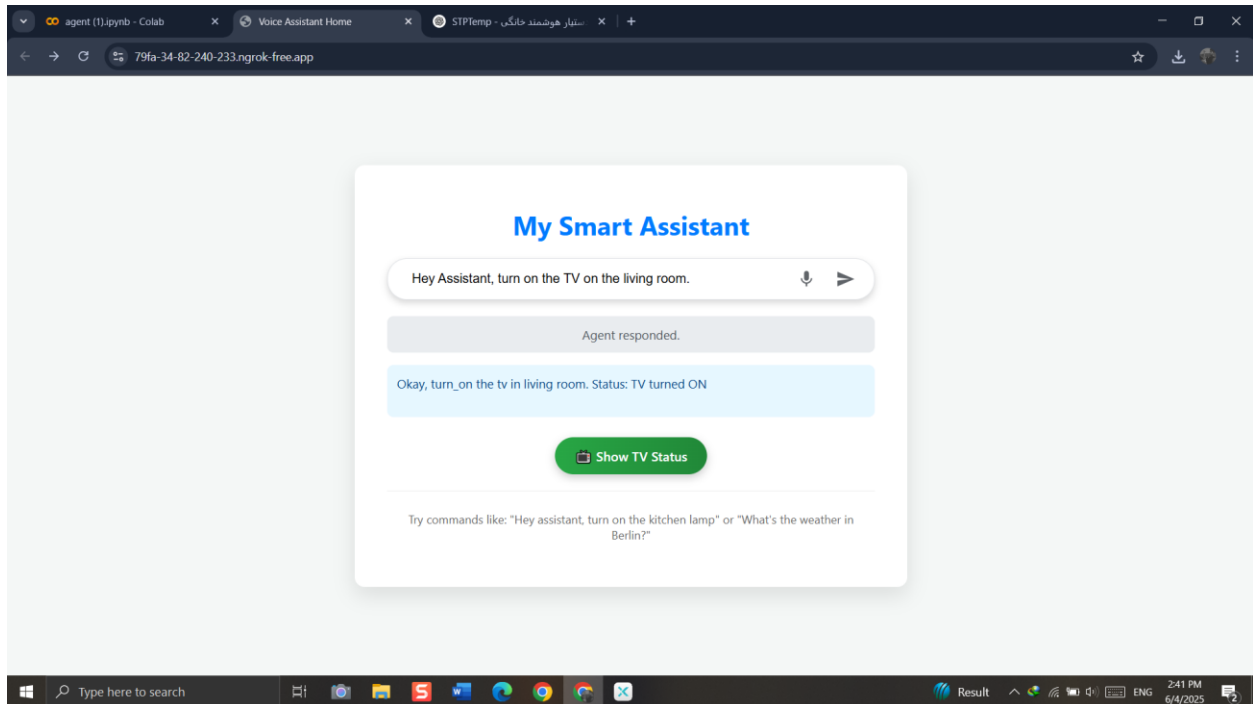
○ فایل صوتی

○ متن ترنسکرایب‌شده برای نمایش در UI

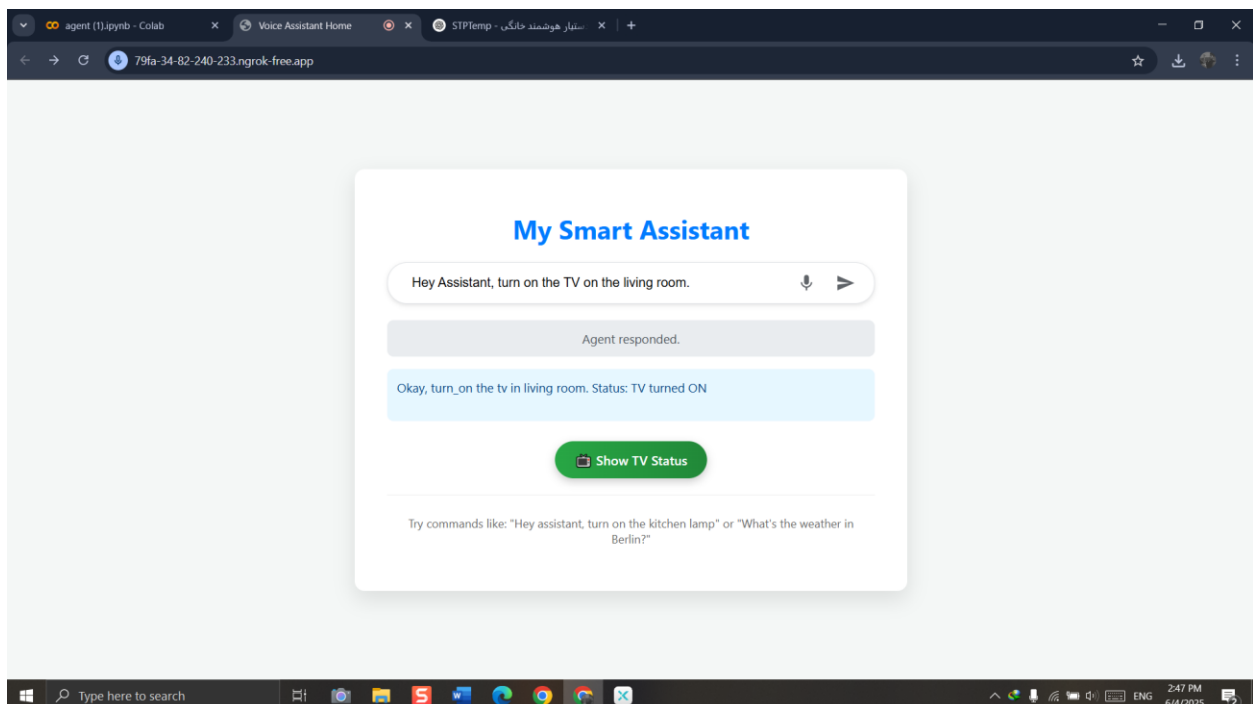
بخش	توضیح
Flask backend	اجرای سرور، تحلیل متن/صدا، تولید پاسخ
ngrok tunnel	تبدیل localhost به لینک اینترنتی برای مرورگر
HTML UI	کادر تایپ، میکروفون، نمایش پاسخ
JS frontend	ضبط صدا، ارسال به بک‌اند، دریافت پاسخ، پخش صوت

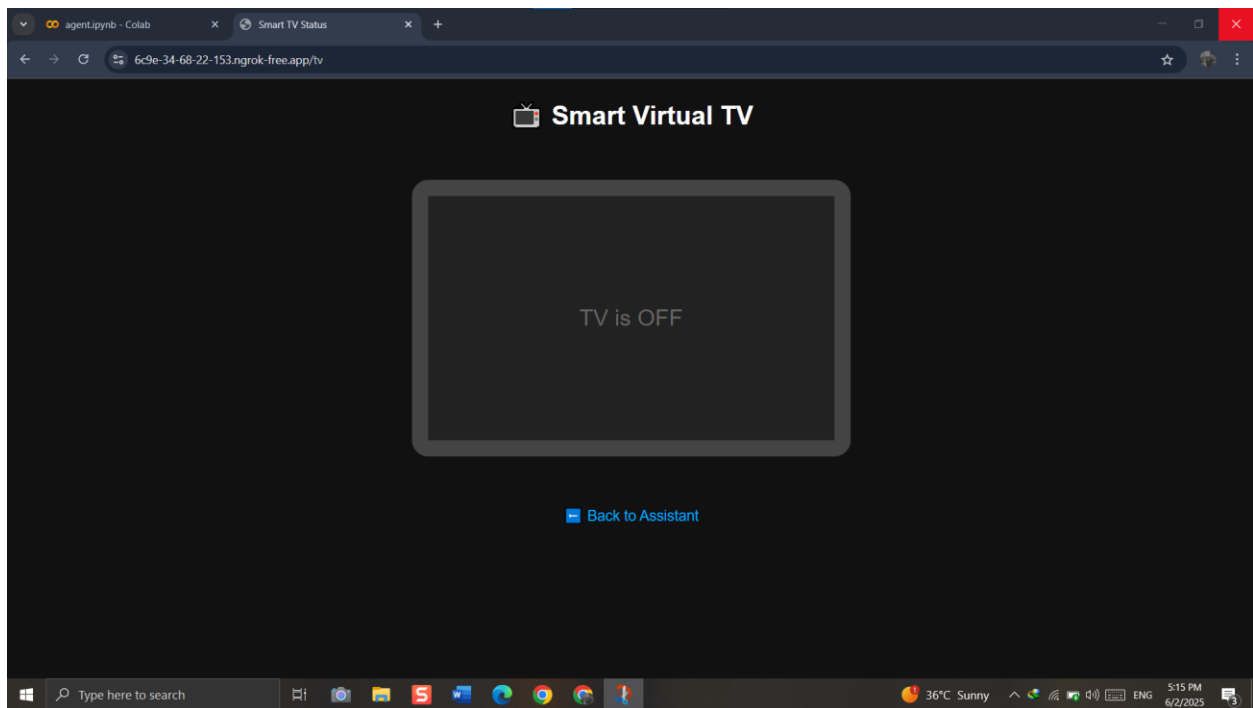
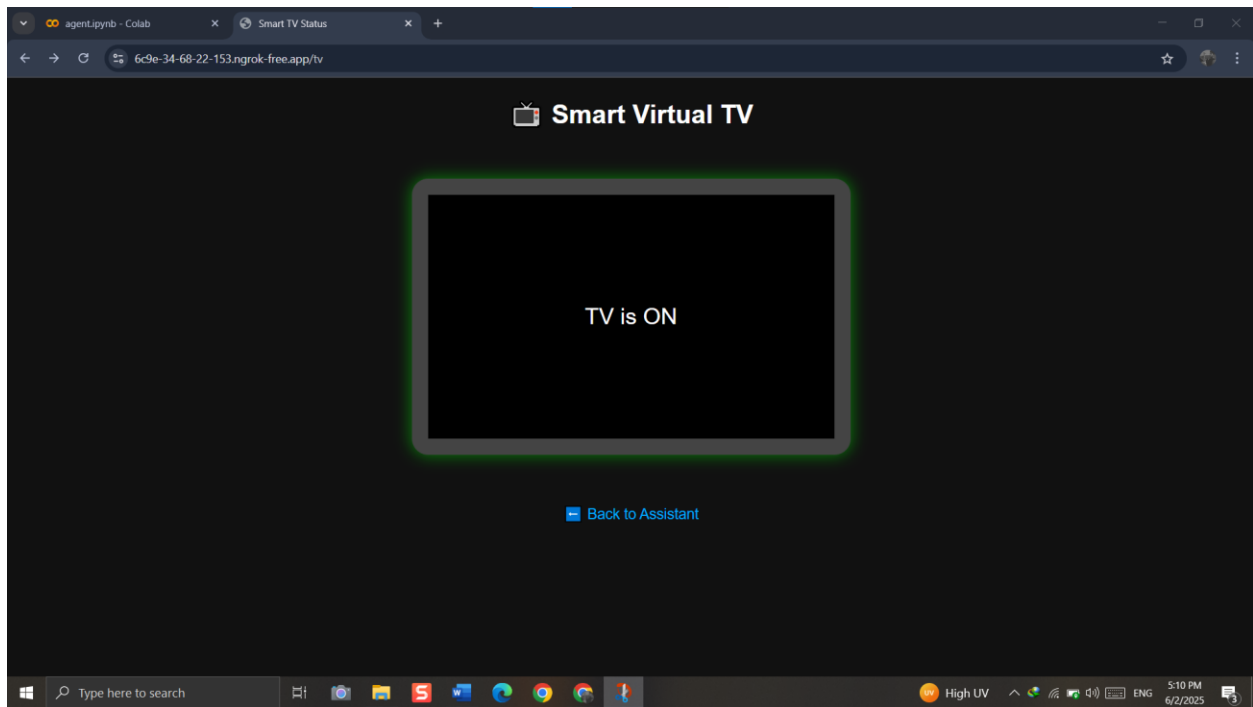
Hey Assistant, turn on/off the TV on the living room.

با پرامپت متنی:

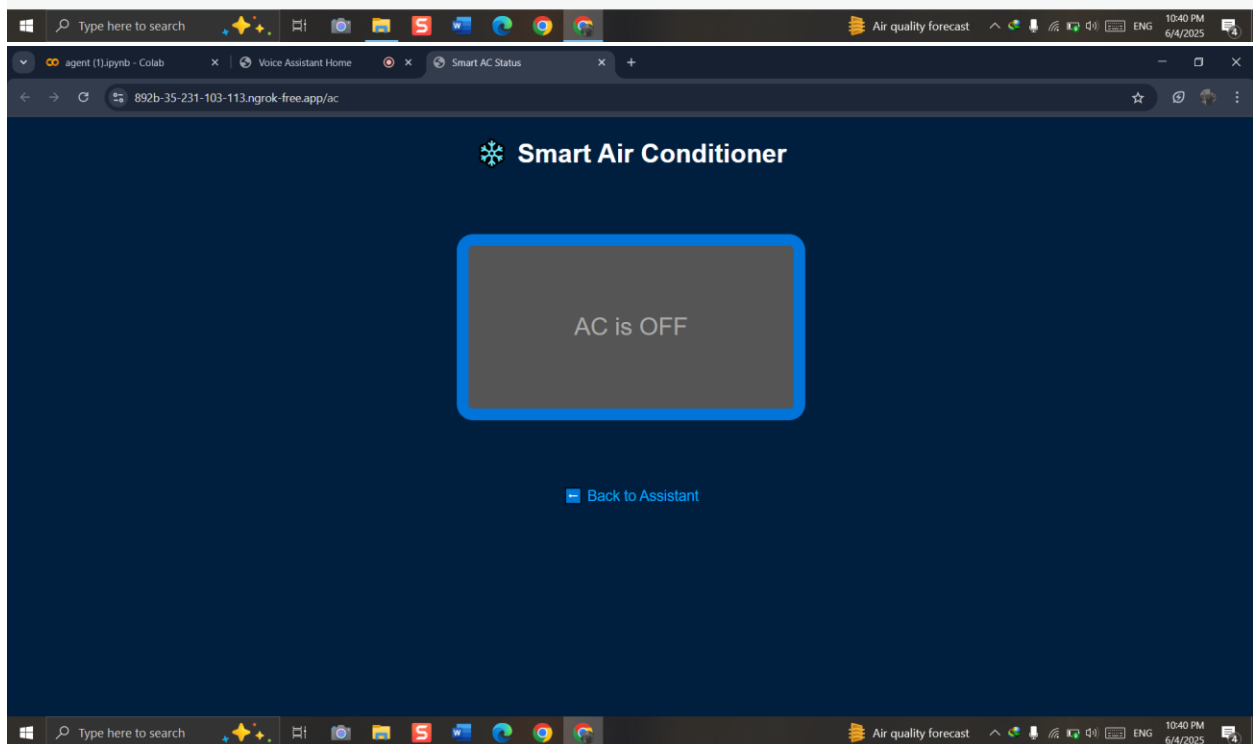
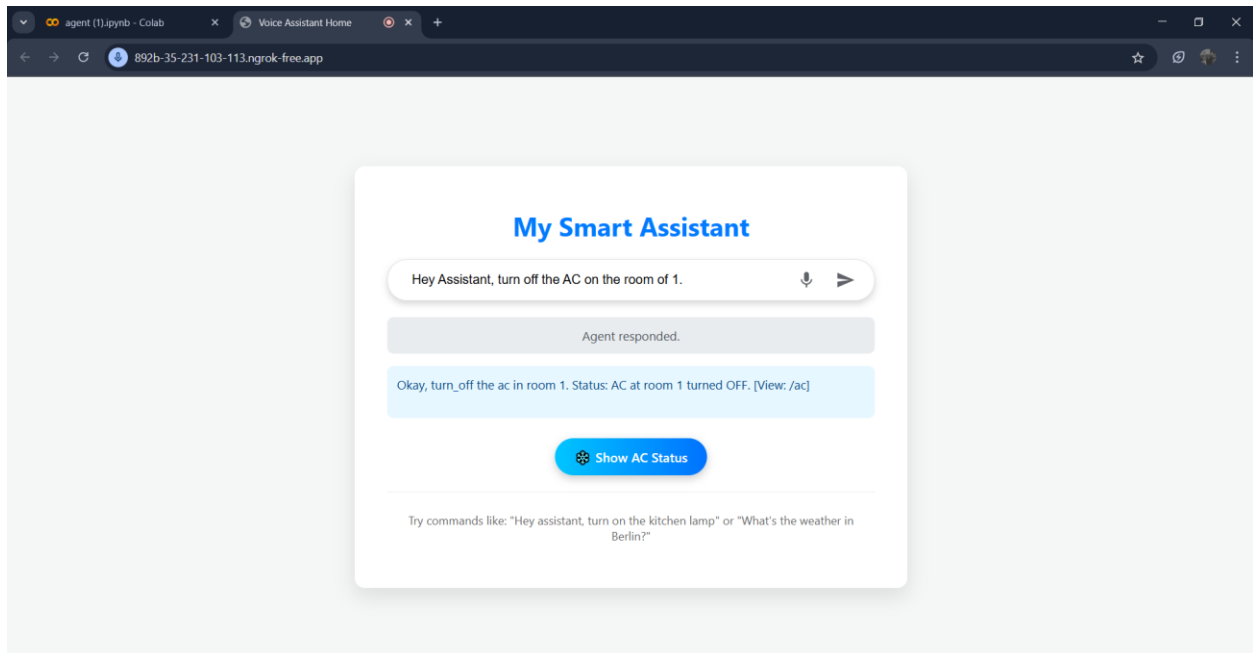


با ویس ریکورد وب:

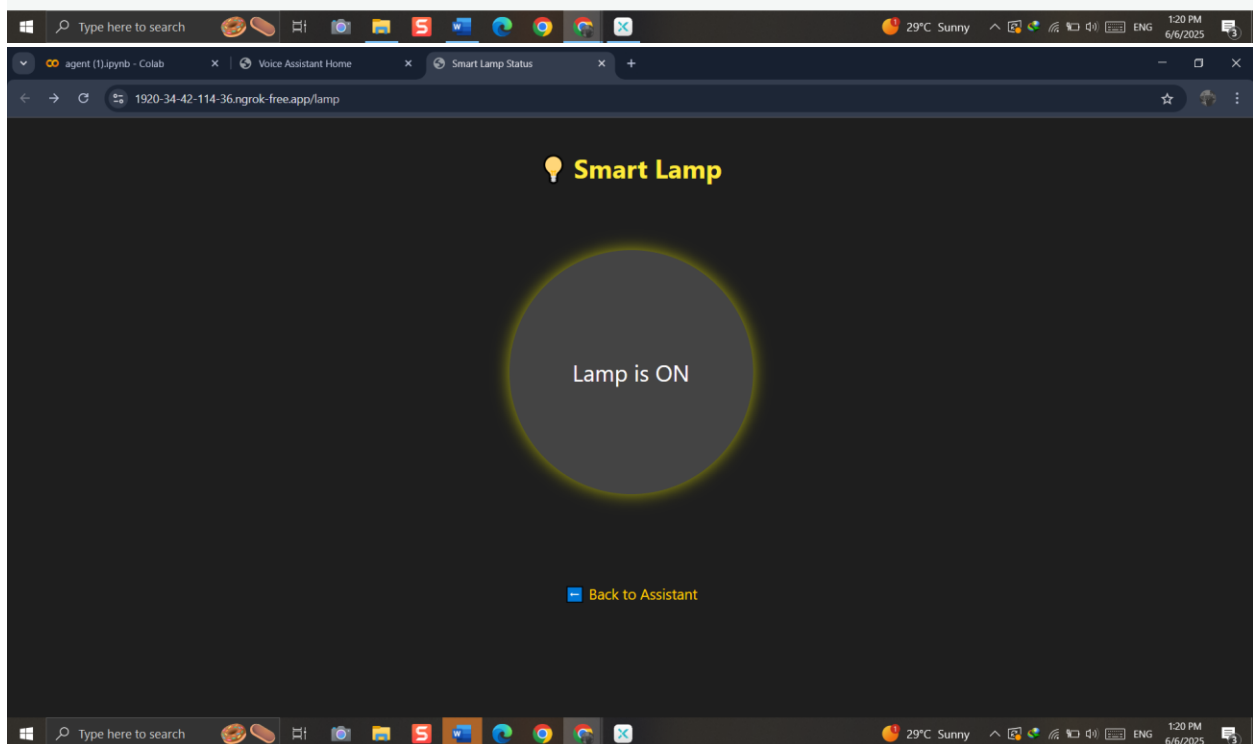
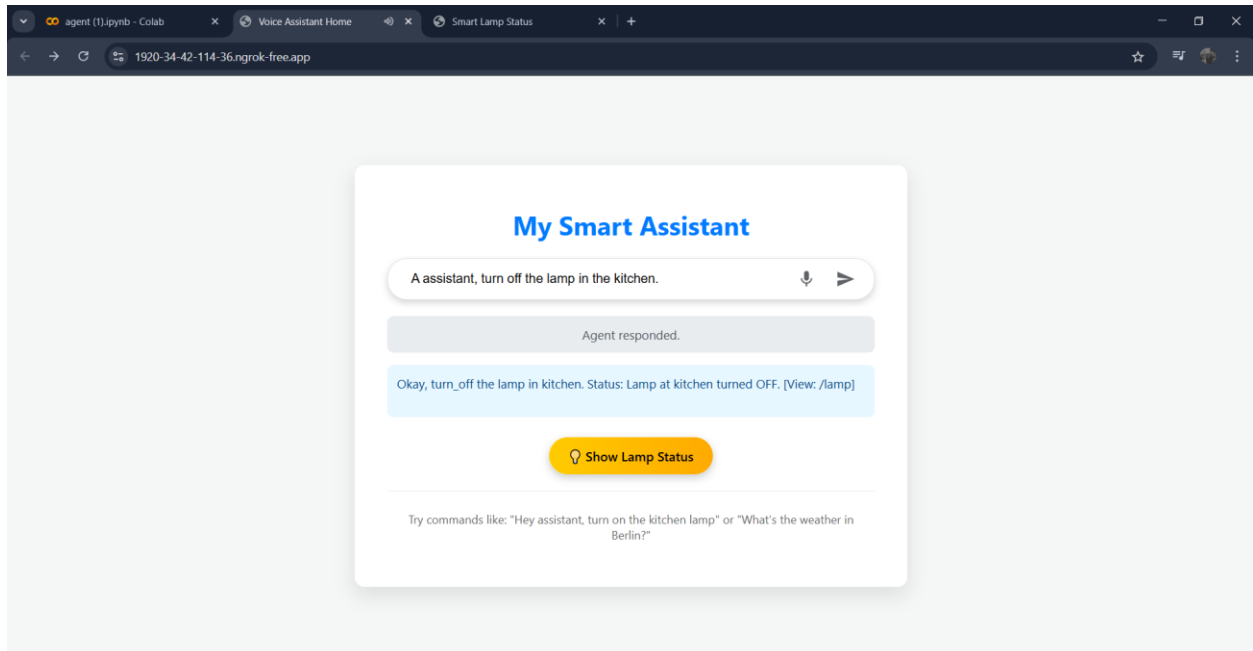




Hey Assistant, turn on/off the AC on the room 1 / Kitchen.



Hey Assistant, turn on/off the lamp on the room 1.



نسخه نهایی و پیشرفته:

